

Logistic Regression: A Powerful Discriminative Model for Classification

Author

Marc'Antonio Lopez

AI & Data Analytics student at Polytechnic University of Turin

Logistic Regression is a fundamental and widely used algorithm categorized as a **discriminative approach** for solving **classification problems**. Unlike generative models (which learn the data distribution for each class, i.e., $P(X | C)$), Logistic Regression directly models the **class posterior probability** $P(C | X)$. Its primary goal is to estimate the probability of an input x belonging to a specific class c , typically expressed as $P(C = c | X = x)$.

Binary Logistic Regression (for Two Classes)

For classification tasks involving exactly two classes (often denoted as h_0 and h_1), Binary Logistic Regression makes a core assumption: it directly posits that the **log-posterior ratio** (also known as **log-odds** or **logit**) is a linear function of the input features x .

This relationship is expressed as:

$$\log \frac{P(C = h_1 | x)}{P(C = h_0 | x)} = w^T x + b$$

Here:

- w is a vector of **weights**, where each element corresponds to a feature in x .
- b is a **bias term** (or intercept).
- $w^T x + b$ represents a **linear combination** of the input features.

Key Characteristic: Linear Decision Boundaries

This assumption of a linear relationship for the log-odds inherently leads to **linear decision boundaries**. This property is also observed in some generative models, such as Gaussian classifiers with tied covariance matrices (which are equivalent to Linear Discriminant Analysis - LDA). A linear decision boundary means the classifier separates classes using a straight line (in 2D feature space) or a flat hyperplane (in higher dimensions).

Deriving Posterior Probability using the Sigmoid Function:

From the linear log-odds relationship, the posterior probability of belonging to class h_1 given input x and parameters (w, b) can be derived using the **sigmoid function** (also known as the logistic function) $\sigma(z)$:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Consequently, the posterior probability for class h_1 is:

$$P(C = h_1 \mid x, w, b) = \sigma(w^T x + b)$$

Function of the Sigmoid:

The sigmoid function is crucial because it transforms any real-valued output from the linear combination $w^T x + b$ (which can range from $-\infty$ to $+\infty$) into a probability value bounded between 0 and 1. A large positive $w^T x + b$ value maps close to 1, while a large negative value maps close to 0.

Training Objective:

The training process of Logistic Regression primarily aims to find the optimal values for the parameters (w, b) that best fit the training data.

Parameter Estimation (Training the Logistic Regression Model)

Training a Logistic Regression model involves finding the optimal parameters (w, b) that best explain the observed training data. This is typically achieved using **Maximum Likelihood Estimation (MLE)**. The training data is assumed to be labeled, independent, and identically distributed (i.i.d.).

The objective function to maximize during training is the **conditional log-likelihood**:

$$\ell(w, b) = \sum_{i=1}^n \log P(C_i = c_i \mid X_i = x_i, w, b)$$

This function measures how probable the observed true class labels c_i are, given the input features x_i and the current model parameters (w, b) .

Equivalence to Minimizing Cross-Entropy Loss:

Maximizing this conditional log-likelihood is mathematically equivalent to minimizing a commonly used loss function called the **binary cross-entropy loss** (also known as log loss):

$$J(w, b) = - \sum_{i=1}^n [c_i \log y_i + (1 - c_i) \log(1 - y_i)]$$

Where:

- c_i is the true binary class label for sample i (e.g., 1 for h_1 , 0 for h_0).
- $y_i = \sigma(w^T x_i + b)$ is the predicted probability for class h_1 for sample i .

Alternative Form (Logistic Loss Function):

When the true labels z_i are encoded as $\{+1, -1\}$ (instead of $\{0, 1\}$), this objective is also known as the **logistic loss function**:

$$\sum_{i=1}^n \log(1 + e^{-z_i(w^T x_i + b)})$$

Optimization Requirement:

Unlike some models (like linear regression or certain generative models) that have closed-form solutions for their parameters, minimizing these objective functions for Logistic Regression **requires numerical optimization algorithms**. Common methods include gradient descent, stochastic gradient descent (SGD), or quasi-Newton methods (like L-BFGS).

Linear Separability and Regularization: Enhancing Model Robustness

Problem with Linearly Separable Data:

A significant issue can arise during Logistic Regression training if the training data is **perfectly linearly separable**. This means a hyperplane exists that can perfectly separate the two classes in the feature space. In such a scenario, the parameters (w, b) can theoretically approach infinity during optimization, leading to several problems:

- **Ill-posed optimization problem:** The optimization objective (log-likelihood) would continue to increase indefinitely, and the algorithm might not converge.
- **Overfitting:** The model becomes overly confident in its predictions for the training data, leading to perfect training accuracy but poor generalization to unseen data.
- **No unique solution:** Multiple hyperplanes could perfectly separate the data, resulting in non-unique solutions for (w, b) .

Solution: L2 Regularization (Ridge Regularization):

To address this issue and enhance model robustness (prevent overfitting and ensure a unique

solution), **L2 regularization** is commonly added to the objective function. L2 regularization adds a penalty term proportional to the squared Euclidean norm of the weight vector w ; the bias term b is typically not regularized.

The new objective function to minimize (combining empirical risk with regularization) becomes:

$$R(w, b) = \underbrace{\frac{1}{n} \sum_{i=1}^n l(z_i(w^T x_i + b))}_{\text{Empirical Risk}} + \underbrace{\frac{\lambda}{2} \|w\|^2}_{\text{L2 Regularization}}$$

Here:

- $l(\cdot)$ represents the individual logistic loss for each sample.
- $\|w\|^2 = \sum_j w_j^2$ is the squared L2 norm of the weight vector.
- λ (lambda) is a crucial **hyper-parameter** (set before training, not learned from data).

Role of λ :

The hyper-parameter λ controls the trade-off between:

- **Fitting the training data well:** A smaller λ allows the model to fit the training data more closely.
- **Keeping the model simple (preventing large weights):** A larger λ penalizes large weights more heavily, forcing them towards zero and creating a simpler model.

Consequences of λ Choice:

- A **high** λ can lead to **underfitting**, where the model is too constrained and cannot effectively capture data patterns.
- A **low** λ (or $\lambda = 0$, meaning no regularization) risks **overfitting**, especially with linearly separable data or noisy features.

Benefits of L2 Regularization:

- Makes the objective function **strictly convex**, ensuring a **unique, finite minimum** for parameters (w, b) , even with linearly separable data.
- Improves the model's **generalization ability** by preventing excessively large weights that are too sensitive to small fluctuations in training data.

Optimal λ Selection: The optimal value for λ is typically chosen through **cross-validation**, where different λ values are tried, and the one yielding the best performance on a validation set is selected.

Feature Preprocessing Recommendation:

It is highly recommended to **preprocess features** by centering them (subtracting the mean) and

scaling them (dividing by standard deviation). This prevents features with naturally large scales from disproportionately dominating the regularization term, ensuring the regularization penalty is applied fairly across all features.

Interpreting Scores and Handling Class Priors

The linear output of the Logistic Regression model, $s(x) = w^T x + b$, represents the **log-posterior ratio** (or log-odds) of class h_1 versus h_0 . This score inherently incorporates the **empirical class priors** (the observed class proportions) present in the training data.

Problem: Prior Mismatch:

If the true **application priors** (the actual class prevalence in the real-world deployment environment, denoted as $\pi_1^{\text{app}}, \pi_0^{\text{app}}$) differ significantly from the empirical priors in the training data ($n_1/n, n_0/n$), then directly using $P(C = 1 | x) = \sigma(s(x))$ for decision-making will be **suboptimal**. The model's probabilities will be skewed based on the training set's composition rather than the real world.

Adjustment for Application Priors:

To properly adjust the scores for the correct application priors, one can "remove" the training prior component from the raw score $s(x)$:

$$s_{\text{llr}}(x) = (w^T x + b) - \log \left(\frac{n_1}{n_0} \right)$$

This adjusted score $s_{\text{llr}}(x)$ now represents a pure **Log-Likelihood Ratio (LLR)**. Decisions can then be made by comparing this $s_{\text{llr}}(x)$ to an optimal Bayes threshold that incorporates the application priors (as discussed in the previous section on Bayes Decisions).

Alternative Prior-Weighting during Training:

Alternatively, if the target application priors are known *before* training, a technique called **prior-weighted logistic regression** can be employed. This involves weighting samples in the training loss function according to the desired application priors, allowing the model to directly optimize for those specific priors.

Performance on MNIST Pairwise Digit Classification (Equal Error Rate - EER):

Feature Type	$\lambda = 0$	$\lambda = 10^{-5}$	$\lambda = 0.001$	$\lambda = 0.1$	Tied Gaussian Model EER
RAW (Full Features)	1.7%	1.4%	1.2%	2.0%	—
PCA (50 Features)	1.4%	1.4%	1.4%	2.1%	1.7%
PCA (9 Features)	1.3%	1.2%	1.2%	2.0%	1.5%

Observations from the MNIST Pairwise Classification Results:

- **Logistic Regression Outperforms Tied Gaussian:** In these pairwise classification tasks, Logistic Regression generally exhibits better performance (lower EER) compared to the Tied Gaussian (LDA) model. This suggests that while both yield linear decision boundaries, Logistic Regression's direct optimization of posterior probabilities is more effective here.
- **Impact of L2 Regularization:** L2 regularization plays a crucial role in improving performance, especially on **RAW (full) features**. For example, with RAW features, adding regularization (from $\lambda = 0$ to $\lambda = 0.001$) reduces the error rate from 1.7% to a notable **1.2%**. This clearly demonstrates regularization's effectiveness in preventing overfitting, even for models with linear decision boundaries.
- **Role of PCA:** While PCA can help reduce feature dimensionality (e.g., from 784 pixels to 50 or 9 components), potentially reducing computational costs and sometimes improving generalization, proper regularization remains essential for achieving optimal performance, regardless of whether raw or PCA-transformed features are used.

Multiclass Logistic Regression (Softmax Regression)

For classification problems involving **more than two classes** ($K > 2$), Logistic Regression is extended to become **Multiclass Logistic Regression**, commonly known as **Softmax Regression**. This extension uses the **softmax function** to estimate the posterior probability for each of the K classes.

The softmax function computes the probability for class k given input x and a separate weight vector w_k and bias b_k for each class:

$$P(C = k \mid x) = \frac{e^{w_k^T x + b_k}}{\sum_{j=1}^K e^{w_j^T x + b_j}}$$

Here:

- w_k and b_k are the weight vector and bias term specifically for class k .
- The numerator $e^{w_k^T x + b_k}$ is the "unnormalized probability" or "score" for class k .
- The denominator $\sum_{j=1}^K e^{w_j^T x + b_j}$ is a normalizing factor that ensures all predicted probabilities for an input x sum to 1 across all K classes.

Training Objective:

Training a Multiclass Logistic Regression model involves minimizing the **multiclass cross-entropy loss** (also known as the negative log-likelihood for multiclass classification):

$$J(W, b) = - \sum_{i=1}^n \sum_{k=1}^K z_{ik} \log y_{ik}$$

Where:

- W represents the matrix of all weight vectors $(w_1, \dots, w_K)$ and b represents the vector of all bias terms $(b_1, \dots, b_K)$.
- z_{ik} is a binary indicator: it is 1 if sample i truly belongs to class k , and 0 otherwise. This is often called "one-hot encoding."
- $y_{ik} = P(C = k \mid x_i)$ is the predicted probability for class k for sample i by the softmax function.

Regularization for Multiclass:

The Multiclass Logistic Regression model is technically **over-parameterized** (meaning there are redundant ways to represent the same decision boundaries). To address this and improve generalization, **L2 regularization** is typically applied to the weight vectors (usually not to the bias terms):

$$\frac{\lambda}{2} \sum_{k=1}^K \|w_k\|^2$$

Decision Boundary Shape:

Similar to its binary counterpart, Multiclass Logistic Regression models inherently learn **linear decision boundaries** between any pair of classes, meaning the feature space is partitioned by a set of hyperplanes.

Multiclass MNIST (10 Digits) Error Rates:

Feature Type	$\lambda = 0$	$\lambda = 10^{-5}$	$\lambda = 0.001$	$\lambda = 0.1$	Tied Gaussian Model Error
RAW (Full Features)	8.0%	7.4%	7.9%	12.9%	—
PCA (50 Features)	8.8%	8.8%	8.9%	13.3%	12.6%
PCA (9 Features)	7.8%	7.8%	8.2%	12.9%	12.3%
PCA+LDA (Combined)	10.9%	10.9%	11.0%	12.4%	12.3%

Observations from the Multiclass MNIST Results:

- **Overall Performance:** Multiclass Logistic Regression generally outperforms the Tied Gaussian model (which is equivalent to LDA) for multiclass classification on MNIST.
- **Crucial Role of Regularization:** Similar to the binary case, regularization proves crucial. For RAW features, adding a small amount of L2 regularization (e.g., $\lambda = 10^{-5}$) significantly reduces the error rate from 8.0% to **7.4%**, effectively preventing overfitting and improving generalization.
- **Dimensionality Reduction Impact:** While dimensionality reduction techniques like PCA and PCA+LDA can be beneficial for reducing feature space and potentially computational costs, they do not always significantly outperform using RAW features when proper regularization is applied to the Logistic Regression model. This highlights the importance of balancing the inherent complexity of the feature set with appropriate model constraints.

Non-linear Classification via Feature Expansion: Going Beyond Linear Boundaries

As discussed, standard Logistic Regression is intrinsically limited to learning **linear decision boundaries**, meaning it can only separate linearly separable classes in the original feature space. To achieve **non-linear decision boundaries**, a powerful and common technique called **feature expansion** (also known as basis expansion or feature transformation) can be employed.

Mechanism of Feature Expansion:

Feature expansion involves transforming the original input feature vector x into an expanded, higher-dimensional feature vector, typically denoted as $\phi(x)$. This transformation explicitly creates new features that are non-linear combinations of the original features.

- **Example: Quadratic Feature Expansion:**

A common example is quadratic feature expansion. If the original features are $x = [x_1, x_2]$, a quadratic expansion $\phi(x)$ would include:

- The original features: x_1, x_2
- Squared terms: x_1^2, x_2^2
- Cross-product terms: $x_1 x_2$

This can be represented compactly as:

$$\phi(x) = [\text{vec}(xx^T); x]$$

Here, $\text{vec}(xx^T)$ flattens the outer product matrix xx^T into a vector, and it's concatenated with the original x .

How it Creates Non-linear Boundaries:

Once the original features are transformed into this higher-dimensional ϕ -space, Logistic Regression (or any linear model) is trained on this expanded vector $\phi(x)$. In this ϕ -space, the model still learns a **linear boundary**. However, when this linear boundary is mapped back to the original x -space, it translates into a **non-linear decision boundary**. For a quadratic expansion, this would result in quadratic decision boundaries (e.g., ellipses, parabolas, hyperbolas).

Challenges with Feature Expansion:

- **Significant Increase in Dimensionality:** The most notable challenge is the dramatic increase in feature space dimensionality. For a quadratic expansion, the number of features can grow by $O(M^2)$ (where M is the original number of features), rapidly escalating computational costs for training and inference.
- **Increased Risk of Overfitting:** With more features, the model has increased flexibility, substantially raising the **risk of overfitting** to the training data. This makes **strong regularization** (e.g., a carefully chosen λ for L2 regularization) even more critical to prevent the model from learning noise.

Other Advanced Non-linear Methods:

Fortunately, other advanced non-linear classification methods exist that achieve non-linear decision boundaries without explicit feature expansion:

- **Support Vector Machines (SVMs) with the kernel trick:** SVMs can implicitly map data into high-dimensional spaces using kernel functions, allowing efficient discovery of non-linear boundaries.
- **Neural Networks:** Deep neural networks are inherently capable of learning complex non-linear relationships and decision boundaries through their multiple layers of non-linear activations.

MNIST Performance with Quadratic Feature Expansion (Applied to PCA Features):

Let's examine the impact of quadratic feature expansion on Logistic Regression performance for MNIST, applied to already PCA-reduced features.

MNIST – Average pairwise EER for LR with quadratic feature expansion

Feature Type (Original)	$\lambda = 0$	$\lambda = 10^{-5}$	$\lambda = 10^{-3}$	$\lambda = 10^{-1}$	Tied Gaussian Model EER
PCA (Features: 50)	1.0%	1.0%	0.9%	1.5%	0.8%

MNIST – Multiclass error rates for LR with quadratic feature expansion

Feature Type (Original)	$\lambda = 0$	$\lambda = 10^{-5}$	$\lambda = 10^{-3}$	$\lambda = 10^{-1}$	Tied Gaussian Model Error
PCA (Features: 50)	2.3%	1.9%	1.7%	3.1%	3.6%

Observations from Quadratic Feature Expansion Results:

- **Dramatic Performance Improvement:** Implementing quadratic feature expansion (after initial PCA) dramatically improves Logistic Regression performance. For example, the multiclass error rate on MNIST drops significantly from 8.8% (for linear LR on PCA-50) to **1.7%**. This clearly demonstrates the substantial benefit of allowing for non-linear decision boundaries when classifying the MNIST dataset.
- **Competitiveness with Gaussian Models:** With this feature expansion, Logistic Regression becomes highly competitive with, or even outperforms, standard Gaussian models (e.g., Full Covariance Gaussian models).
- **Regularization Remains Critical:** Consistent with other Logistic Regression setups, regularization (tuning λ) remains absolutely critical for achieving optimal results with feature expansion. The very low error rates (e.g., 0.9% EER, 1.7% multiclass error) are achieved at intermediate λ values, highlighting the need to balance model complexity introduced by expansion with regularization.